

Database Engineering

Lecture 4: Logical Database Design and the Relational Model

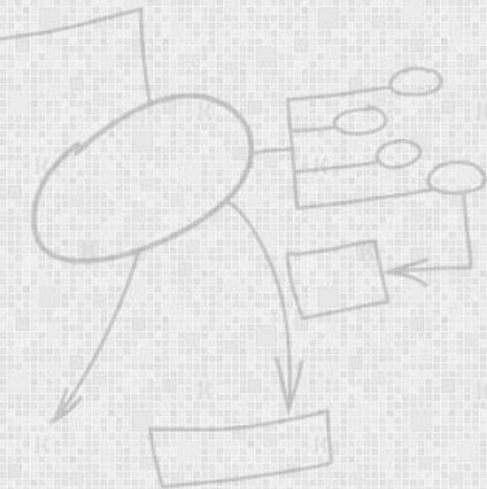
By

Dr Wasi Haider Butt



Agenda

- Properties of relations
- Properties of candidate keys
- Entity and relational integrity constraints
- Transformation of E-R and EER diagrams to relations



Relation

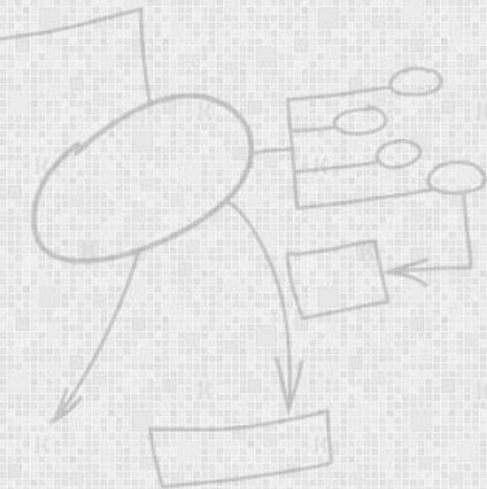
- Definition: A relation is a named, two-dimensional table of data
- Table consists of rows (records) and columns (attribute or field)
- Requirements for a table to qualify as a relation:
 - It must have a unique name
 - Every attribute value must be atomic (not multivalued, not composite)
 - Every row must be unique (can't have two rows with exactly the same values for all their fields)
 - Attributes (columns) in tables must have unique names
 - The order of the columns must be irrelevant
 - The order of the rows must be irrelevant

NOTE: all *relations* are in **1st Normal form**



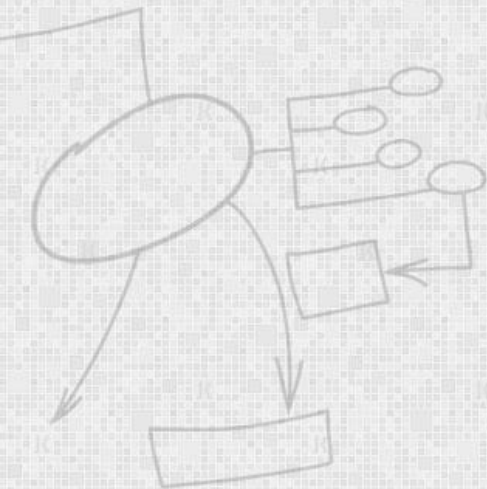
Degree and Cardinality of Relation

- ***Degree: No of columns***
- ***Cardinality: No of rows***

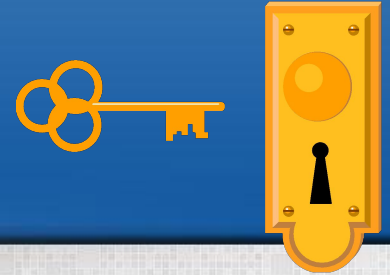


Correspondence with E-R Model

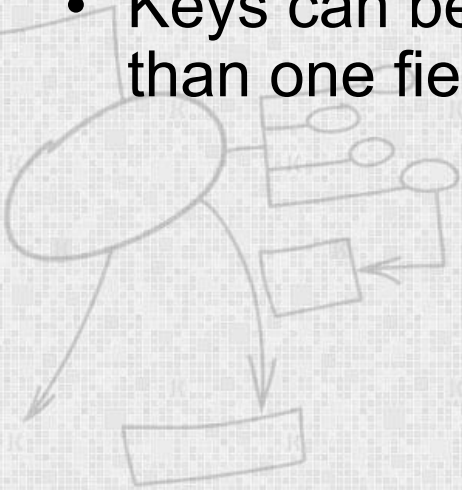
- Relations (tables) correspond with entity types
- Rows correspond with entity instances
- Columns correspond with attributes



Key Fields



- Keys are special fields that serve two main purposes:
 - **Primary keys** are unique identifiers of the relation in question. Examples include employee numbers, social security numbers, etc. *This is how we can guarantee that all rows are unique*
 - **Foreign keys** are identifiers that enable a dependent relation (on the many side of a relationship) to refer to its parent relation (on the one side of the relationship)
- Keys can be **simple** (a single field) or **composite** (more than one field)



Schema for four relations (Pine Valley Furniture Company)

CUSTOMER					
<u>Customer_ID</u>	Customer_Name	Customer_Address	City *	State *	Postal_Code *

Primary Key

ORDER		
<u>Order_ID</u>	Order_Date	<u>Customer_ID</u>

Foreign Key

(implements 1:N relationship between customer and order)

ORDER LINE		
<u>Order_ID</u>	<u>Product_ID</u>	Ordered_Quantity

Combined, these are a *composite primary key* (uniquely identifies the order line)...individually they are *foreign keys* (implement M:N relationship between order and product)

PRODUCT				
<u>Product_ID</u>	Product_Description	Product_Finish	Standard_Price	Product_Line_ID

* Not in Figure 3-22 for simplicity.



Integrity Constraints

- Domain Constraints
 - Allowable values for an attribute.
- Entity Integrity
 - No primary key attribute may be null. All primary key fields **MUST** have data
- Referential Integrity

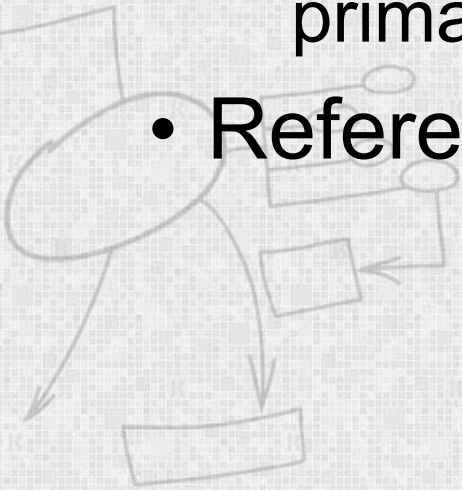


Table 5-1 Domain Definitions for INVOICE Attributes

<i>Attribute</i>	<i>Domain Name</i>	<i>Description</i>	<i>Domain</i>
Customer_ID	Customer_IDs	Set of all possible customer IDs	character: size 5
Customer_Name	Customer_Names	Set of all possible customer names	character: size 25
Customer_Address	Customer_Addresses	Set of all possible customer addresses	character: size 30
City	Cities	Set of all possible cities	character: size 20
State	States	Set of all possible states	character: size 2
Postal_Code	Postal_Codes	Set of all possible postal zip codes	character: size 10
Order_ID	Order_IDs	Set of all possible order IDs	character: size 5
Order_Date	Order_Dates	Set of all possible order dates	date format mm/dd/yy
Product_ID	Product_IDs	Set of all possible product IDs	character: size 5
Product_Description	Product_Descriptions	Set of all possible product descriptions	character size 25
Product_Finish	Product_Finishes	Set of all possible product finishes	character: size 15
Standard_Price	Unit_Prices	Set of all possible unit prices	monetary: 6 digits
Product_Line_ID	Product_Line_IDs	Set of all possible product line IDs	integer: 3 digits
Ordered_Quantity	Quantities	Set of all possible ordered quantities	integer: 3 digits

Domain definitions enforce domain integrity constraints

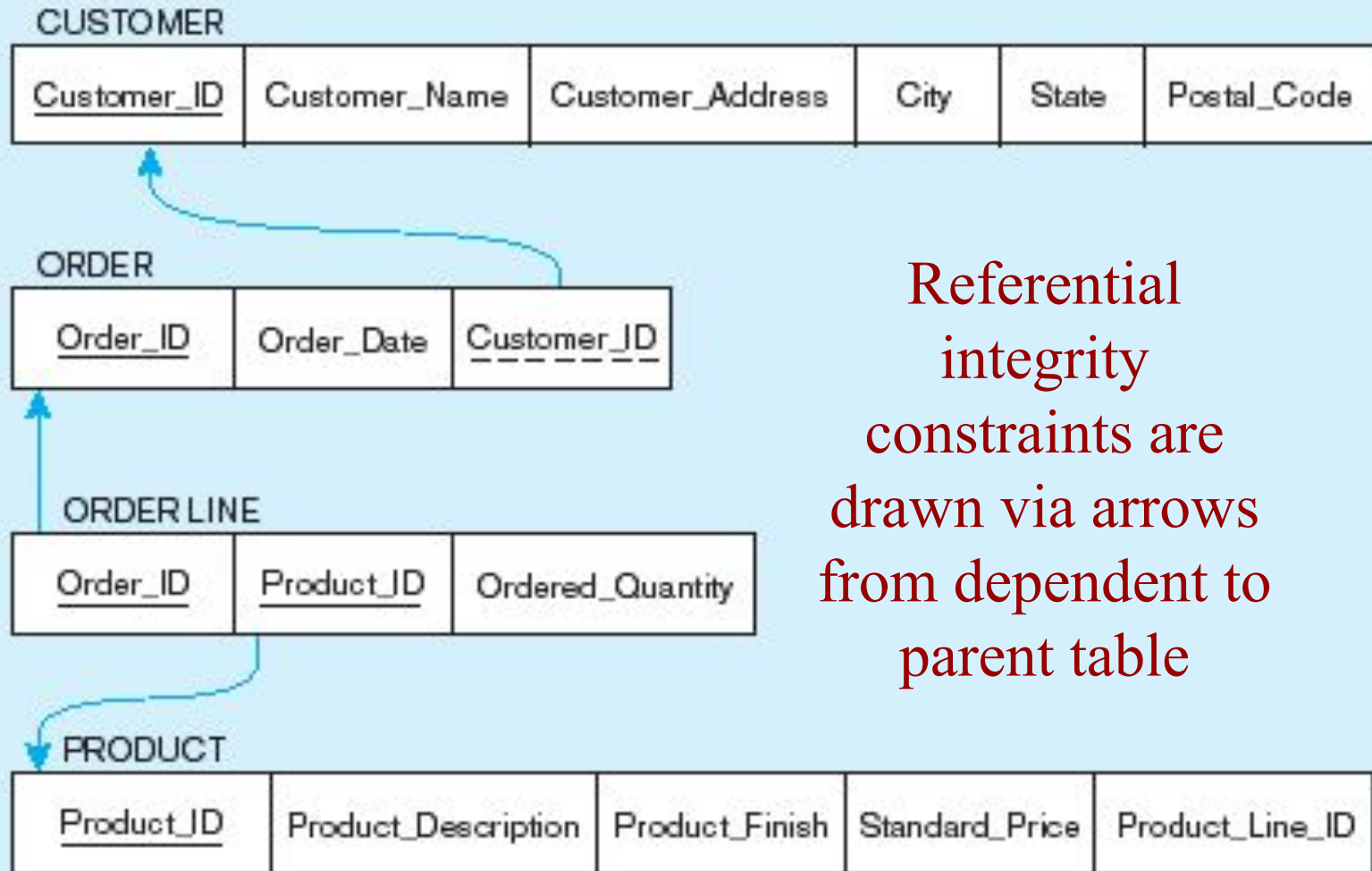


Integrity Constraints

- Referential Integrity—rule states that any foreign key value (on the relation of the dependent side) **MUST** match a primary key value in the relation of the parent side. (Or the foreign key can be null)
 - Delete Rules
 - Restrict—don't allow delete of “parent” side if related rows exist in “dependent” side
 - Cascade—automatically delete “dependent” side rows that correspond with the “parent” side row to be deleted
 - Set-to-Null—set the foreign key in the dependent side to null if deleting from the parent side ☐ not allowed for weak entities



Referential integrity constraints (Pine Valley Furniture)



Referential
integrity
constraints are
drawn via arrows
from dependent to
parent table

SQL table definitions

```
CREATE TABLE CUSTOMER
(CUSTOMER_ID          VARCHAR(5)          NOT NULL,
CUSTOMER_NAME        VARCHAR(25)        NOT NULL,
CUSTOMER ADDRESS     VARCHAR(30)        NOT NULL,
CITY                 VARCHAR(20)        NOT NULL,
STATE               CHAR(2)             NOT NULL,
POSTAL_CODE         CHAR(10)           NOT NULL,
PRIMARY KEY (CUSTOMER_ID);

CREATE TABLE ORDER
(ORDER_ID            CHAR(5)            NOT NULL,
ORDER DATE          DATE               NOT NULL,
CUSTOMER_ID        VARCHAR(5)          NOT NULL,
PRIMARY KEY (ORDER_ID),
FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER (CUSTOMER_ID);

CREATE TABLE ORDER_LINE
(ORDER_ID           CHAR(5)            NOT NULL,
PRODUCT_ID         CHAR(5)            NOT NULL,
ORDERED_QUANTITY   INT               NOT NULL,
PRIMARY KEY (ORDER_ID, PRODUCT_ID),
FOREIGN KEY (ORDER_ID) REFERENCES ORDER (ORDER_ID),
FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT (PRODUCT_ID);

CREATE TABLE PRODUCT
(PRODUCT_ID         CHAR(5)            NOT NULL,
PRODUCT_DESCRIPTION VARCHAR(25),
PRODUCT_FINISH     VARCHAR(12),
STANDARD_PRICE     DECIMAL(8,2)       NOT NULL,
PRODUCT_LINE_ID    INT               NOT NULL,
PRIMARY KEY (PRODUCT_ID);
```

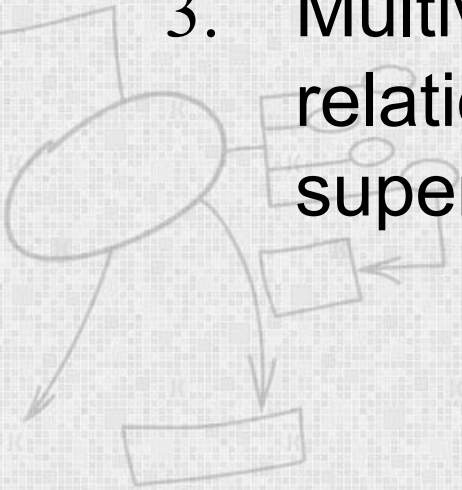
Referential
integrity
constraints are
implemented with
foreign key to
primary key
references



Transforming EER Diagrams into Relations

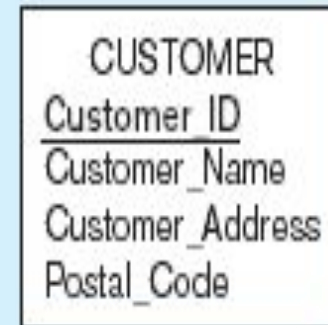
Mapping Regular Entities to Relations

1. Simple attributes: E-R attributes map directly onto the relation
2. Composite attributes: Use only their simple, component attributes
3. Multivalued Attribute—Becomes a separate relation with a foreign key taken from the superior entity

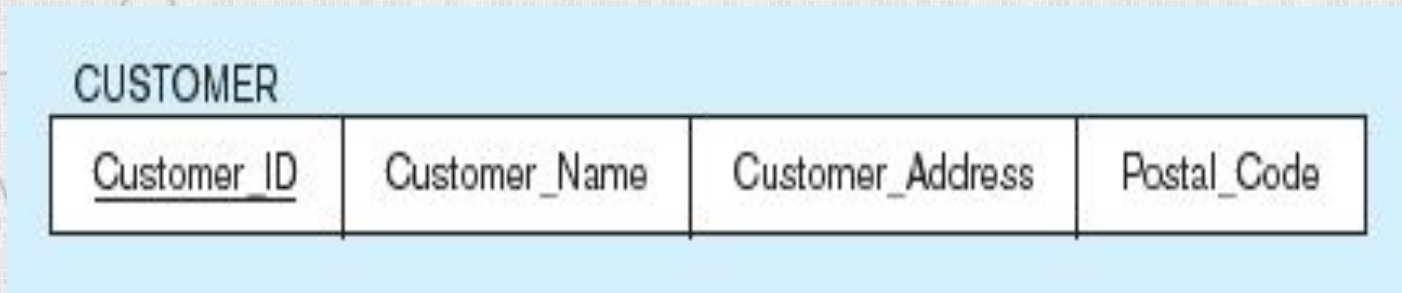


Mapping a regular entity

(a) CUSTOMER entity type with simple attributes



(b) CUSTOMER relation

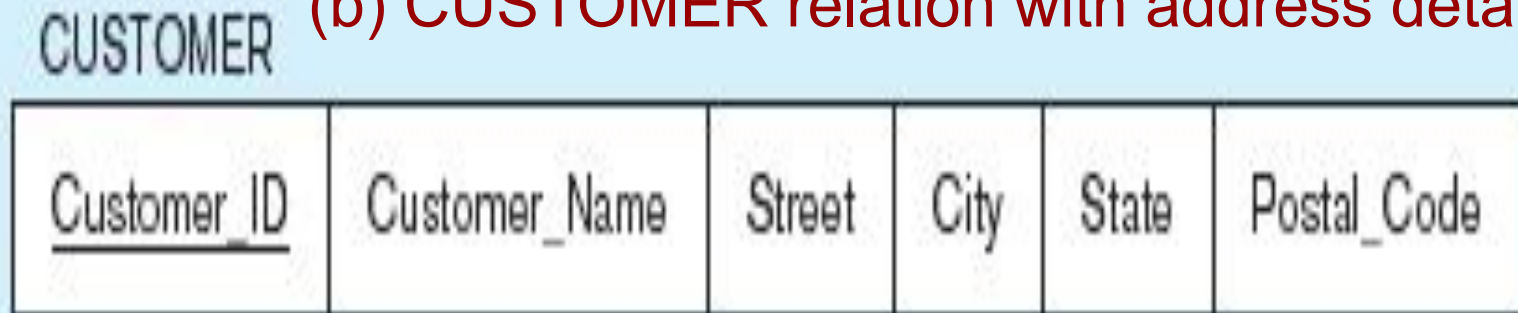


Mapping a composite attribute

(a) CUSTOMER
entity type with
composite
attribute

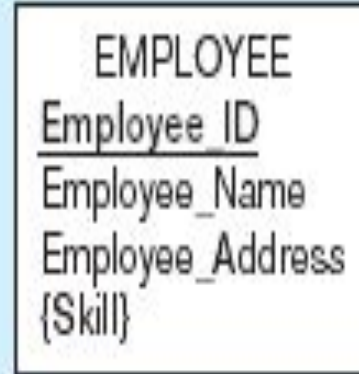


(b) CUSTOMER relation with address detail



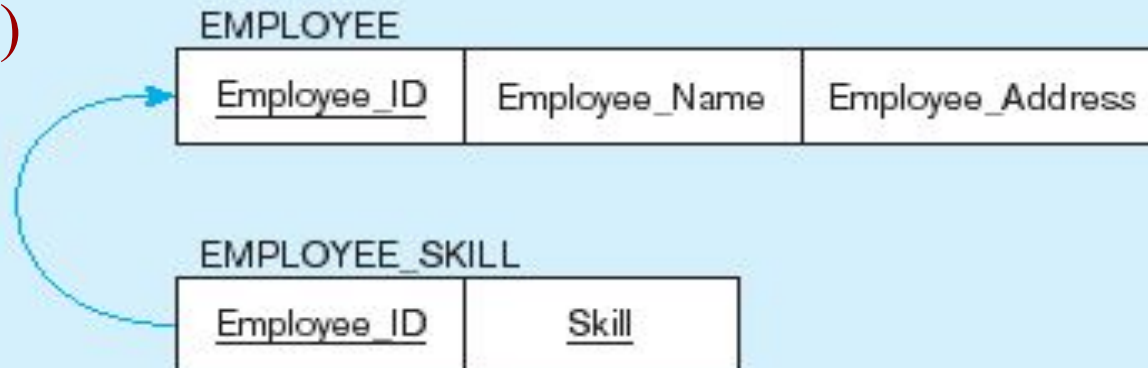
Mapping an entity with a multivalued attribute

(a)



Multivalued attribute becomes a separate relation with foreign key

(b)



One-to-many relationship between original entity and new relation

Transforming EER Diagrams into Relations (cont.)

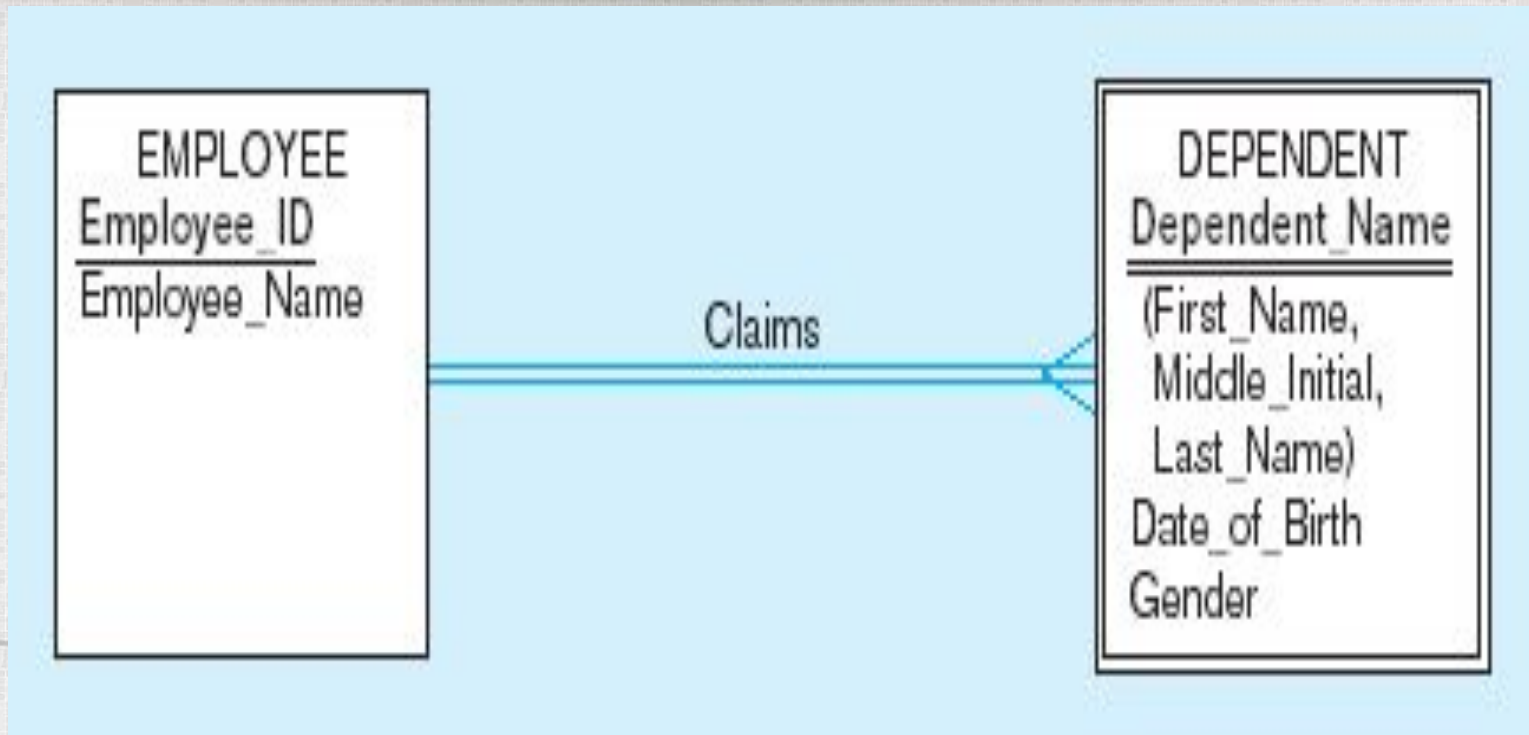
Mapping Weak Entities

- Becomes a separate relation with a foreign key taken from the superior entity
- Primary key composed of:
 - Partial identifier of weak entity
 - Primary key of identifying relation (strong entity)



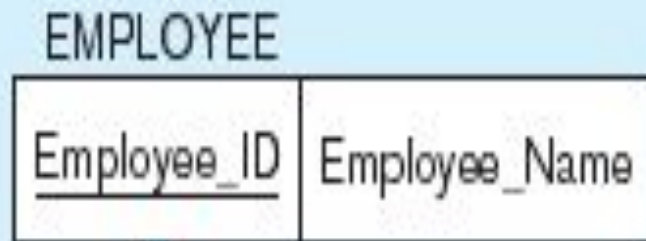
Example of mapping a weak entity

a) Weak entity DEPENDENT

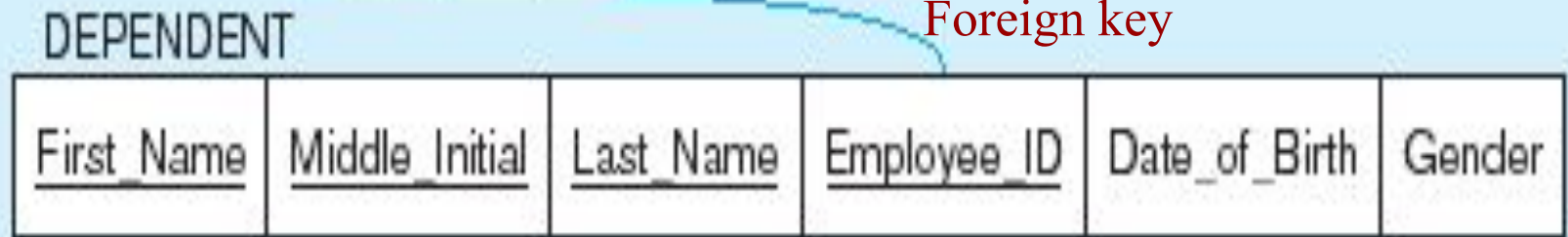


Example of mapping a weak entity (cont.)

b) Relations resulting from weak entity



NOTE: the domain constraint for the foreign key should NOT allow *null* value if DEPENDENT is a weak entity



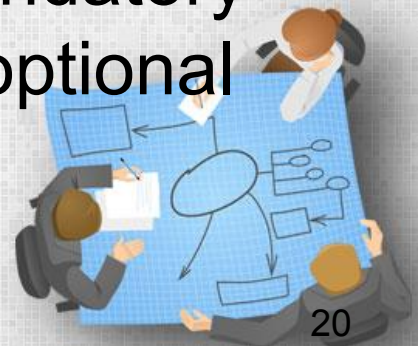
Composite primary key



Transforming EER Diagrams into Relations (cont.)

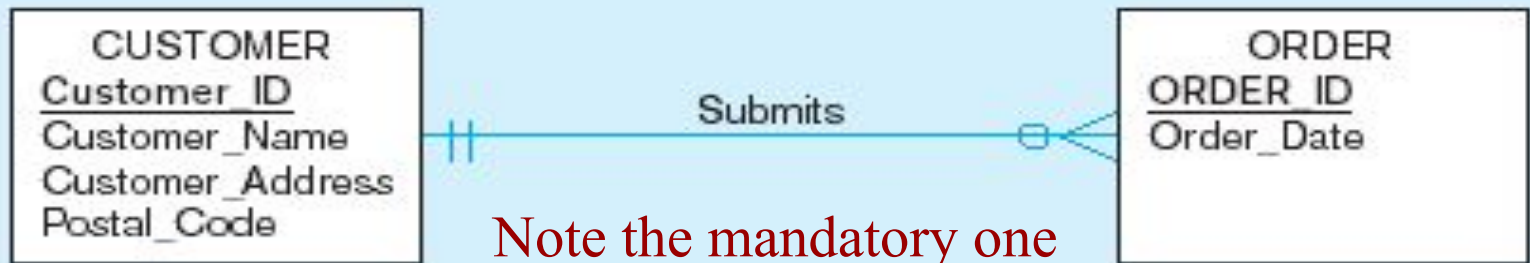
Mapping Binary Relationships

- One-to-Many—Primary key on the one side becomes a foreign key on the many side
- Many-to-Many—Create a ***new relation*** with the primary keys of the two entities as its primary key
- One-to-One—Primary key on the mandatory side becomes a foreign key on the optional side

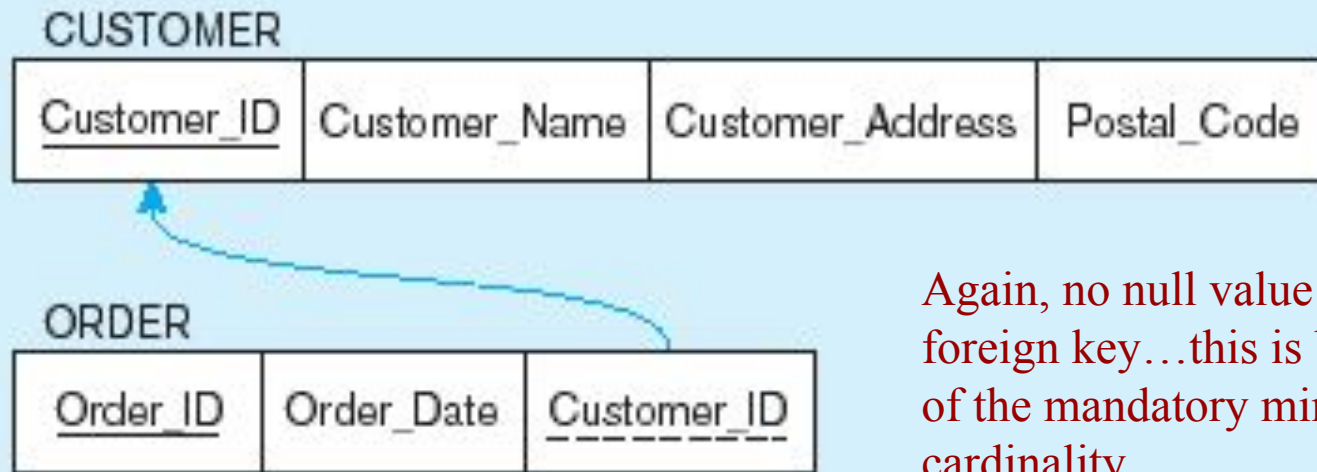


Example of mapping a 1:M relationship

a) Relationship between customers and orders



b) Mapping the relationship

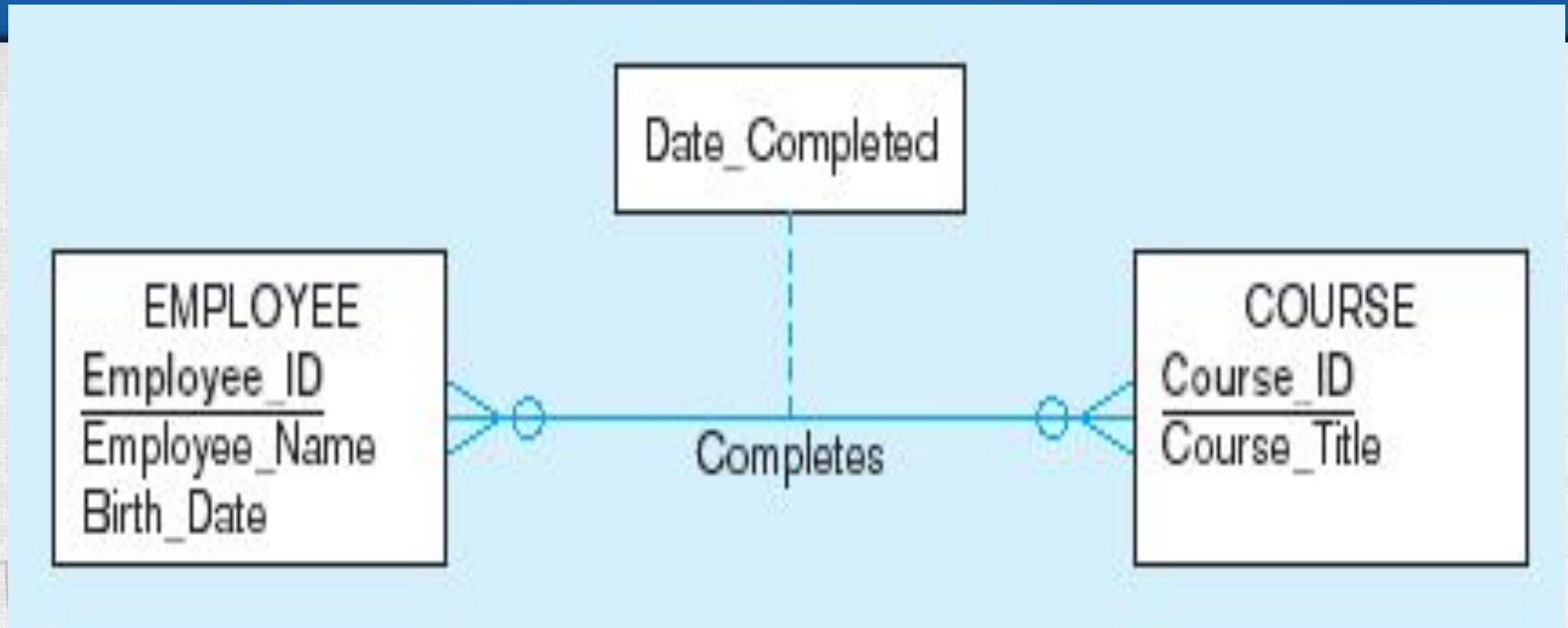


Foreign key

Again, no null value in the foreign key...this is because of the mandatory minimum cardinality

Example of mapping an M:N relationship

a) Completes relationship (M:N)

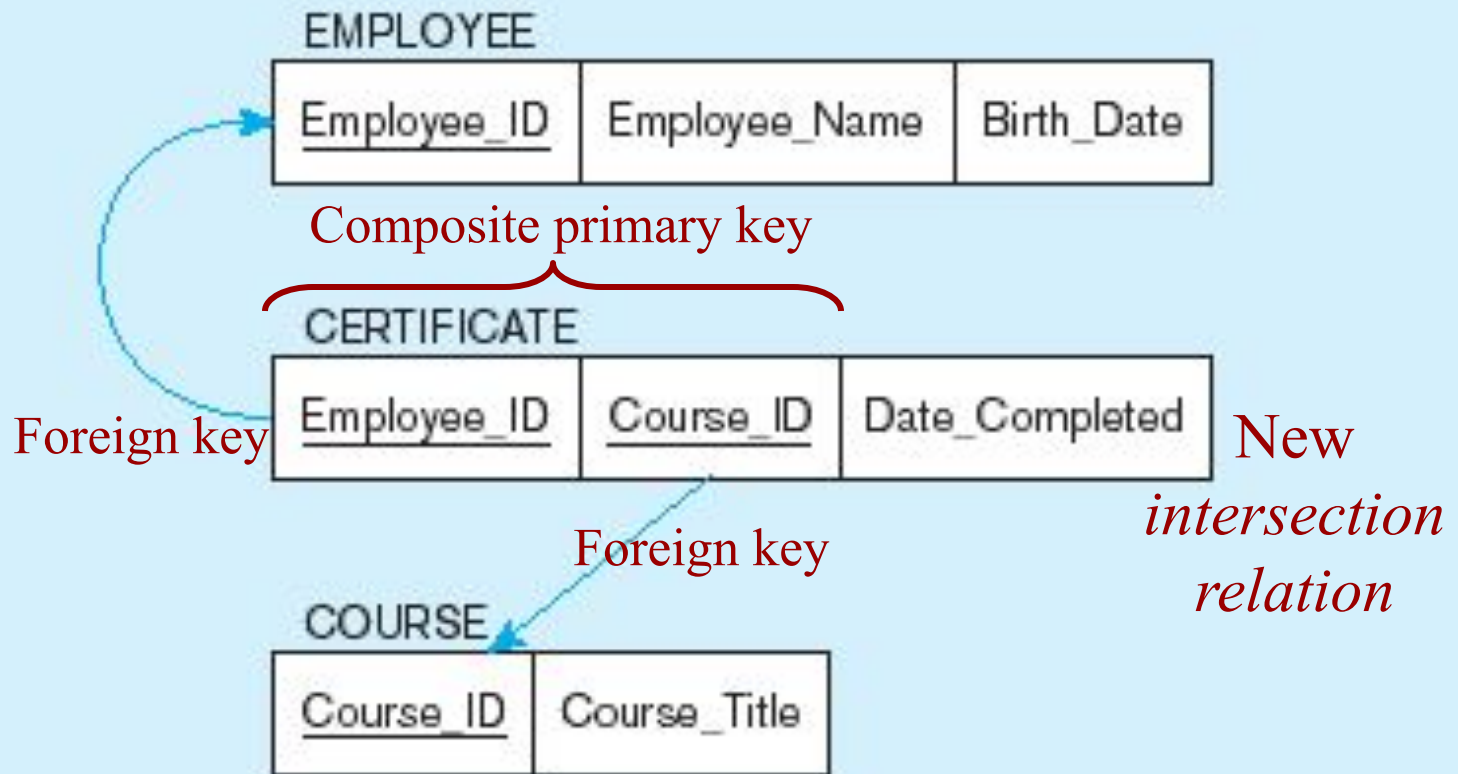


The *Completes* relationship will need to become a separate relation



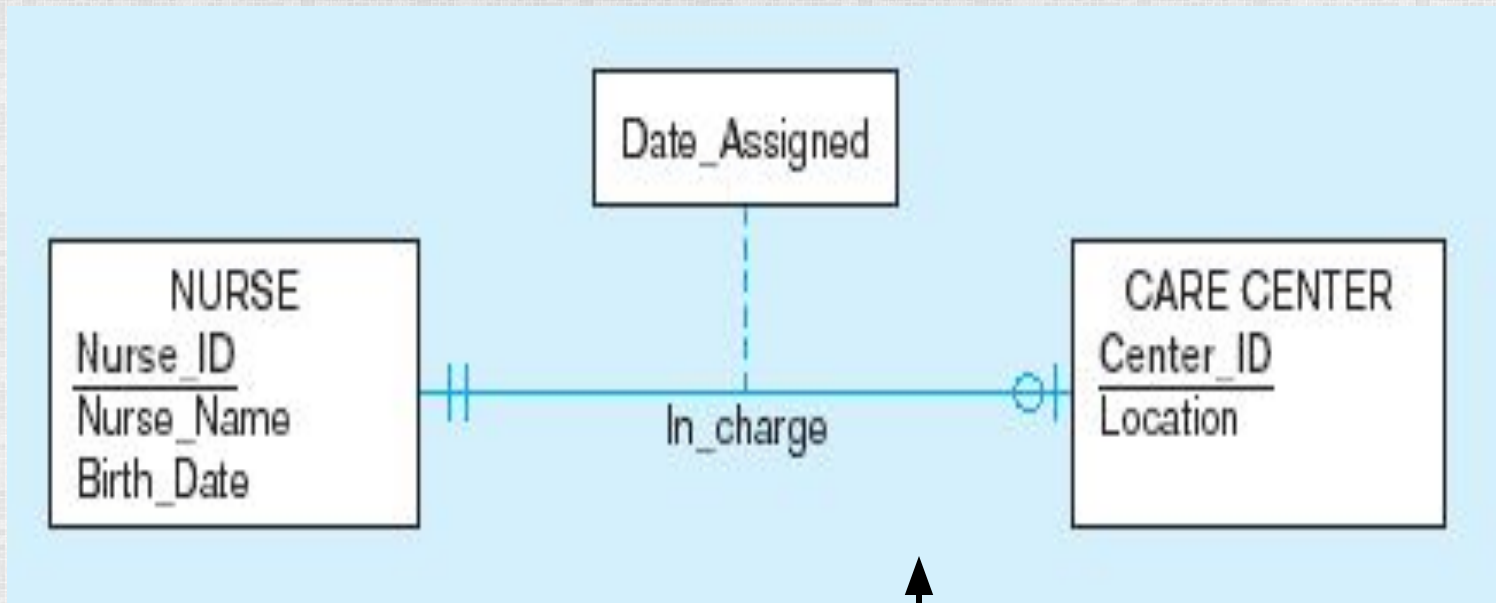
Example of mapping an M:N relationship (cont.)

b) Three resulting relations



Example of mapping a binary 1:1 relationship

a) In_charge relationship (1:1)

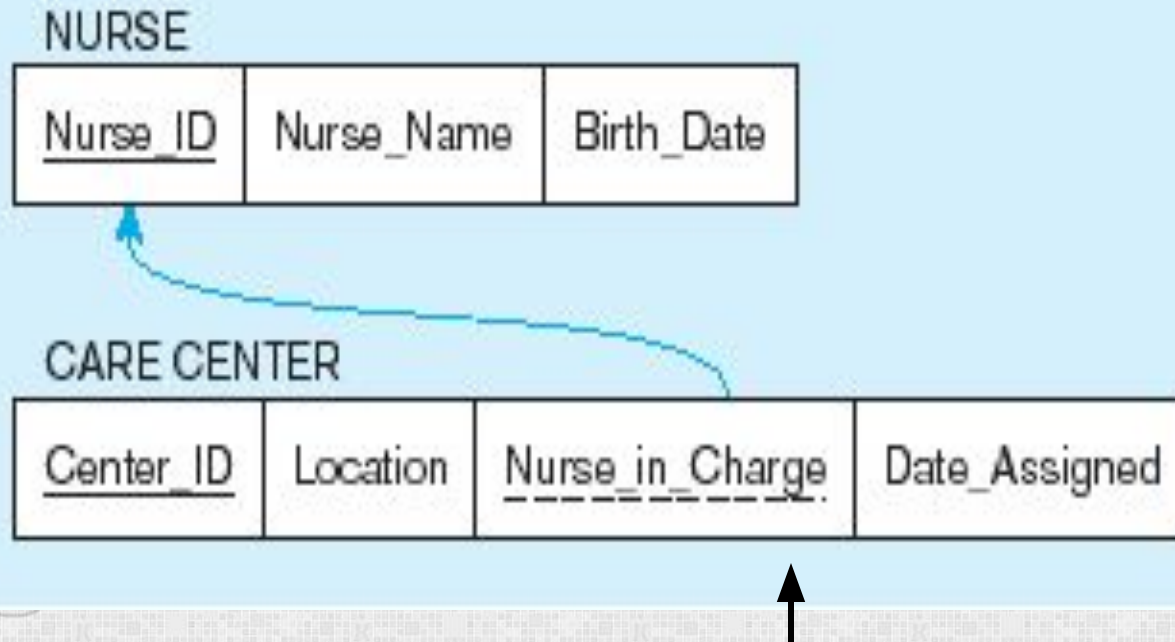


Often in 1:1 relationships, one direction is optional.



Example of mapping a binary 1:1 relationship (cont.)

b) Resulting relations



Foreign key goes in the relation on the optional side,
Matching the primary key on the mandatory side



Transforming EER Diagrams into Relations (cont.)

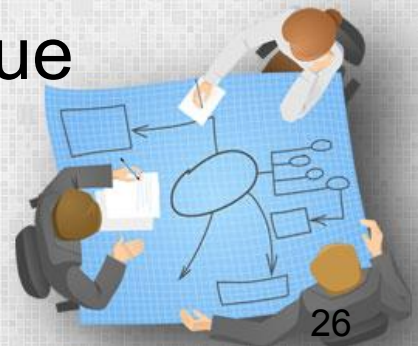
Mapping Associative Entities

– Identifier Not Assigned

- Default primary key for the association relation is composed of the primary keys of the two entities (as in M:N relationship)

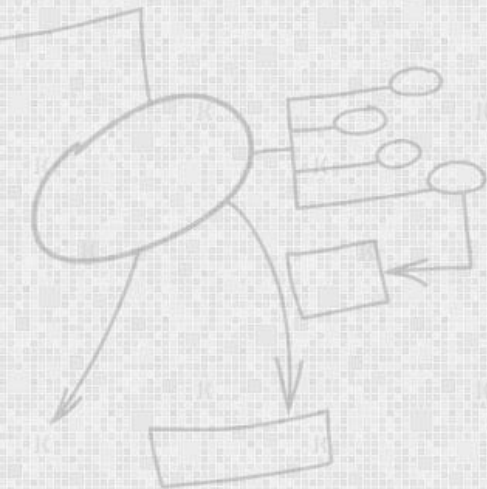
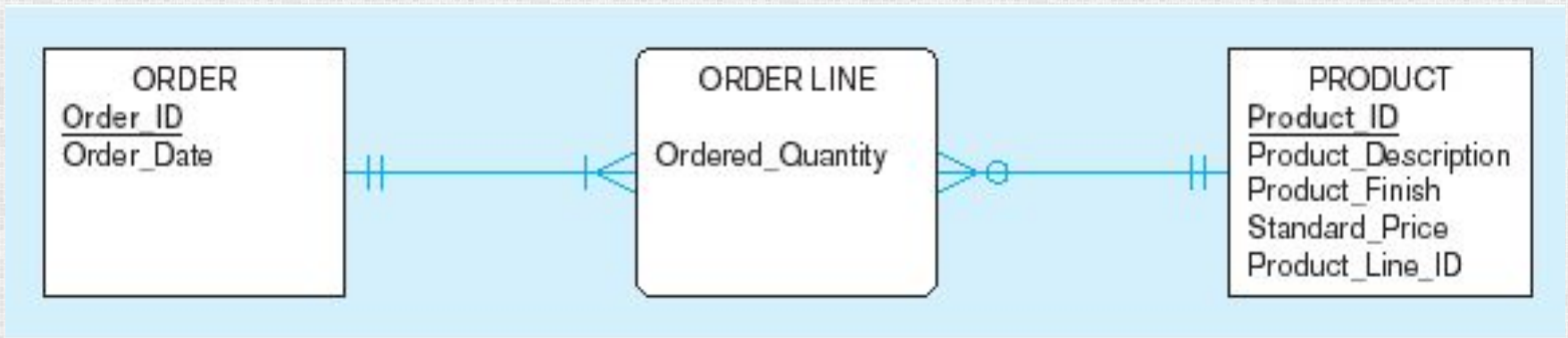
– Identifier Assigned

- It is natural and familiar to end-users
- Default identifier may not be unique



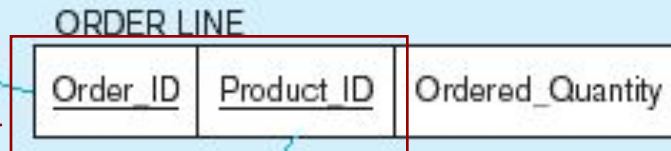
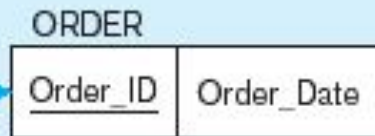
Example of mapping an associative entity

a) An associative entity



Example of mapping an associative entity (cont.)

b) Three resulting relations



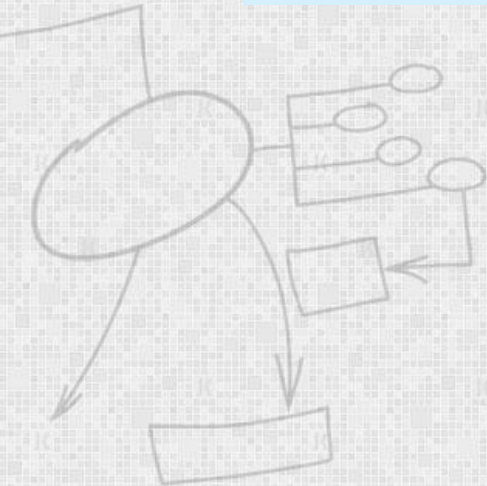
Note: Product_Line_ID is included here because it is a foreign key into the PRODUCT LINE entity, not because it would normally be included as an attribute of PRODUCT

Composite primary key formed from the two foreign keys



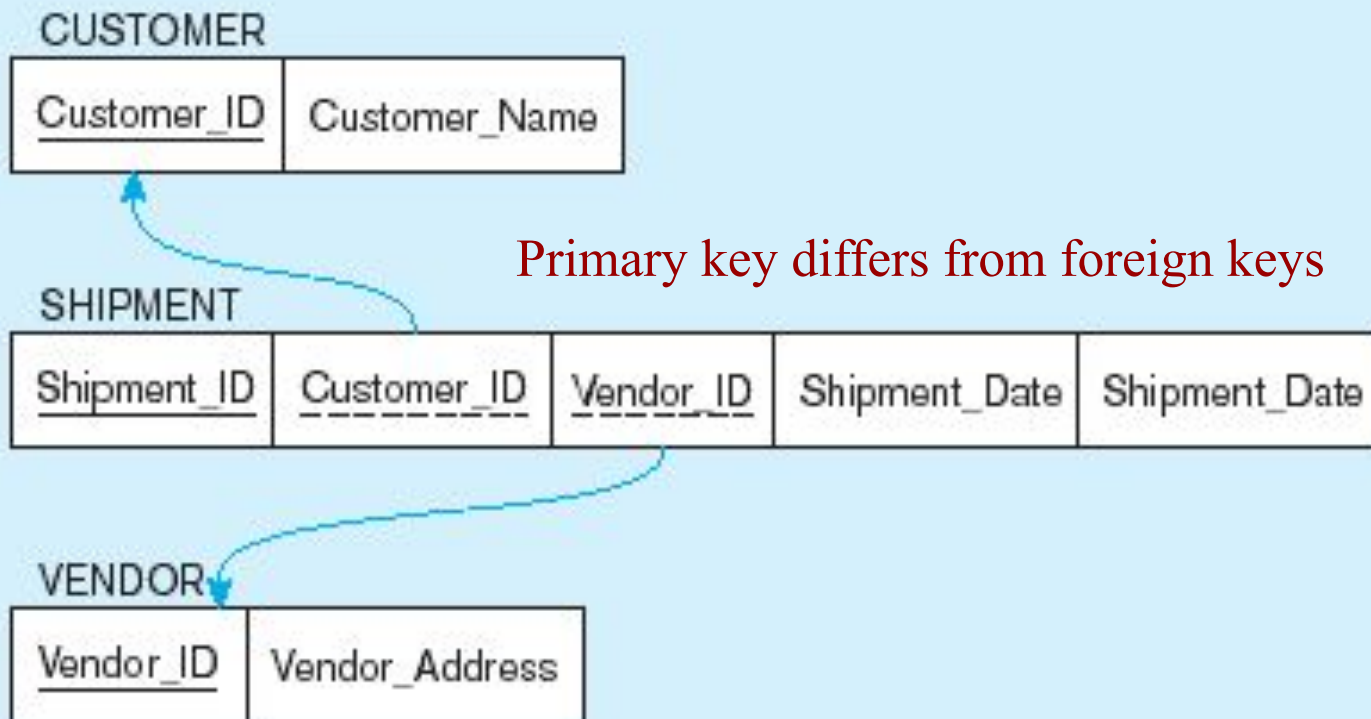
Example of mapping an associative entity with an identifier

a) SHIPMENT associative entity



Example of mapping an associative entity with an identifier (cont.)

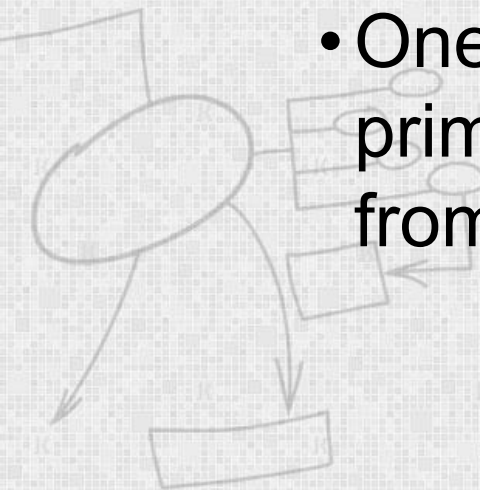
b) Three resulting relations



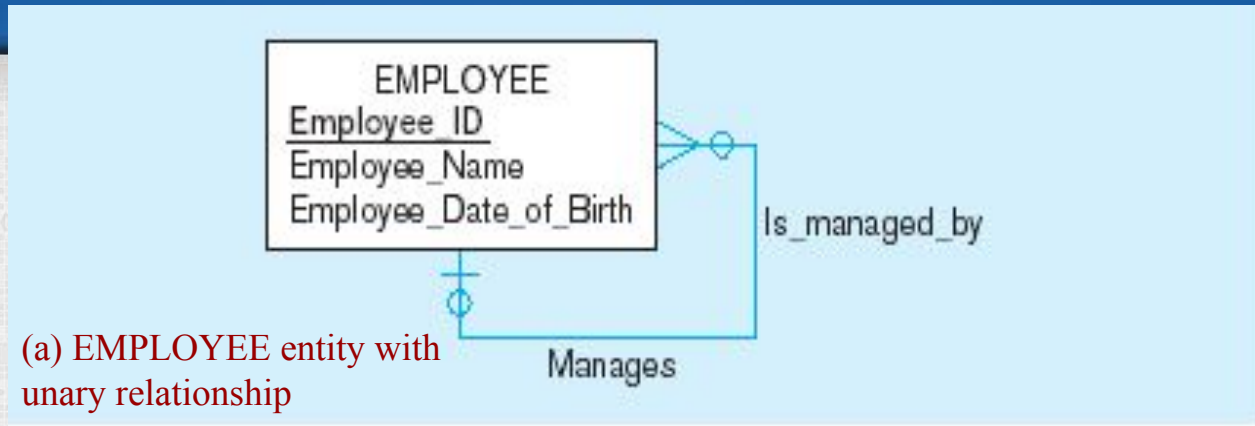
Transforming EER Diagrams into Relations (cont.)

Mapping Unary Relationships

- One-to-Many–Recursive foreign key in the same relation
- Many-to-Many–Two relations:
 - One for the entity type
 - One for an associative relation in which the primary key has two attributes, both taken from the primary key of the entity



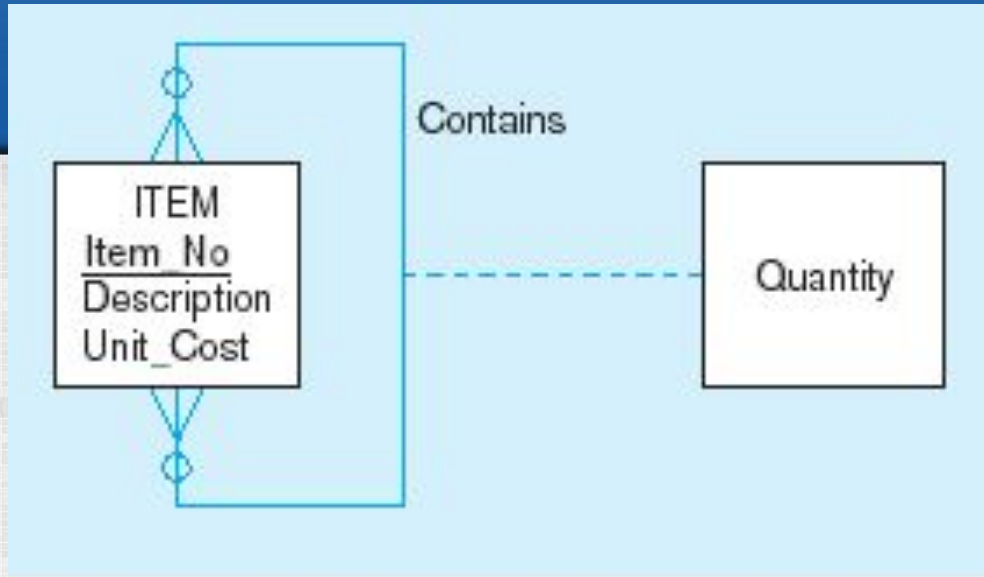
Mapping a unary 1:N relationship



(b) EMPLOYEE relation with recursive foreign key

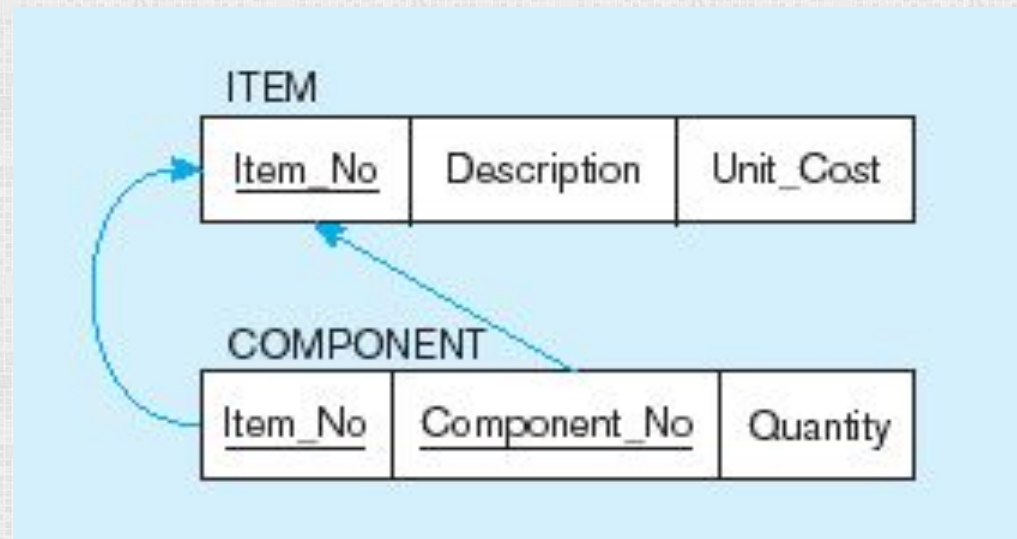


Mapping a unary M:N relationship



(a) Bill-of-materials relationships (M:N)

(b) ITEM and COMPONENT relations



Transforming EER Diagrams into Relations (cont.)

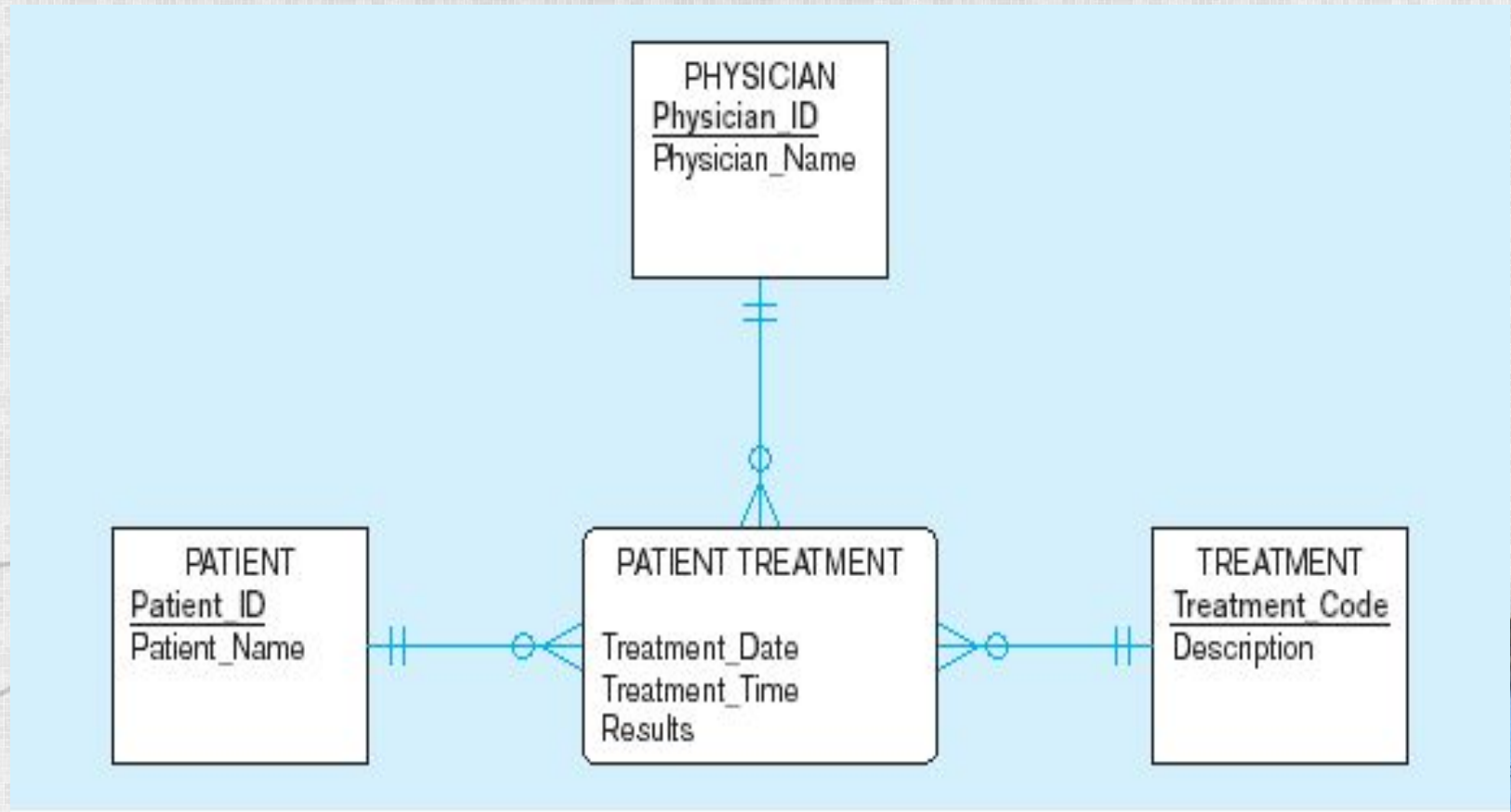
Mapping Ternary (and n-ary) Relationships

- One relation for each entity and one for the associative entity
- Associative entity has foreign keys to each entity in the relationship



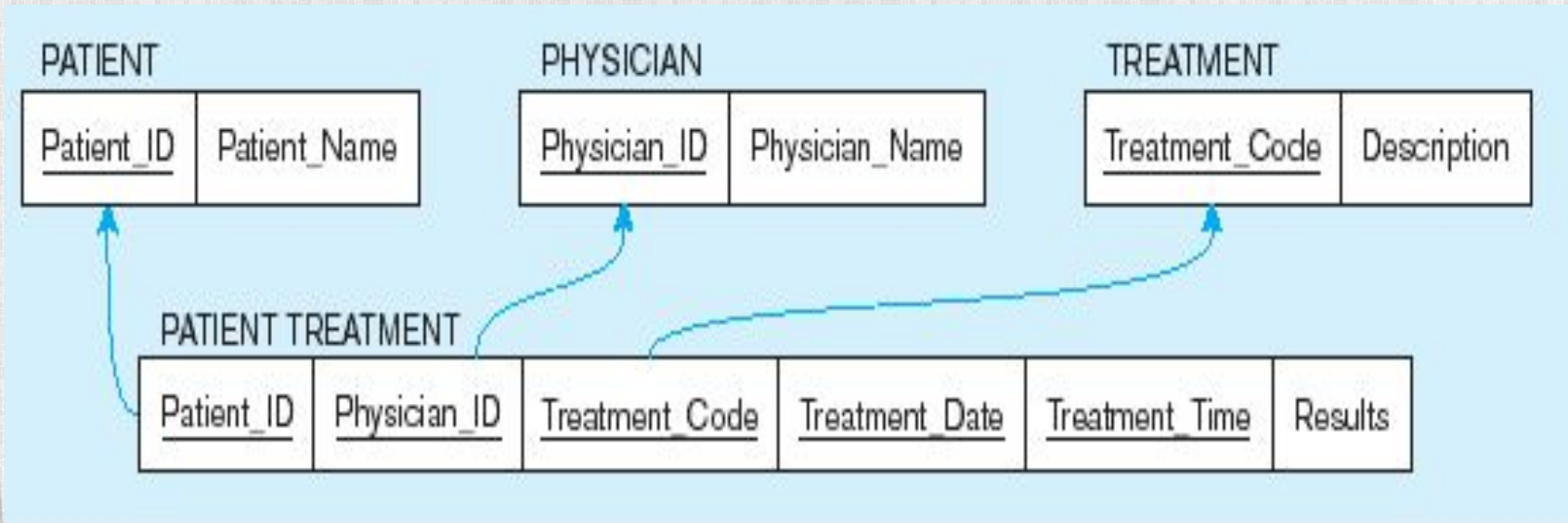
Mapping a ternary relationship

a) PATIENT TREATMENT Ternary relationship with associative entity



Mapping a ternary relationship (cont.)

b) Mapping the ternary relationship PATIENT TREATMENT



Remember that the primary key **MUST** be unique

This is why treatment date and time are included in the composite primary key

But this makes a very cumbersome key...

It would be better to create a surrogate key like Treatment#



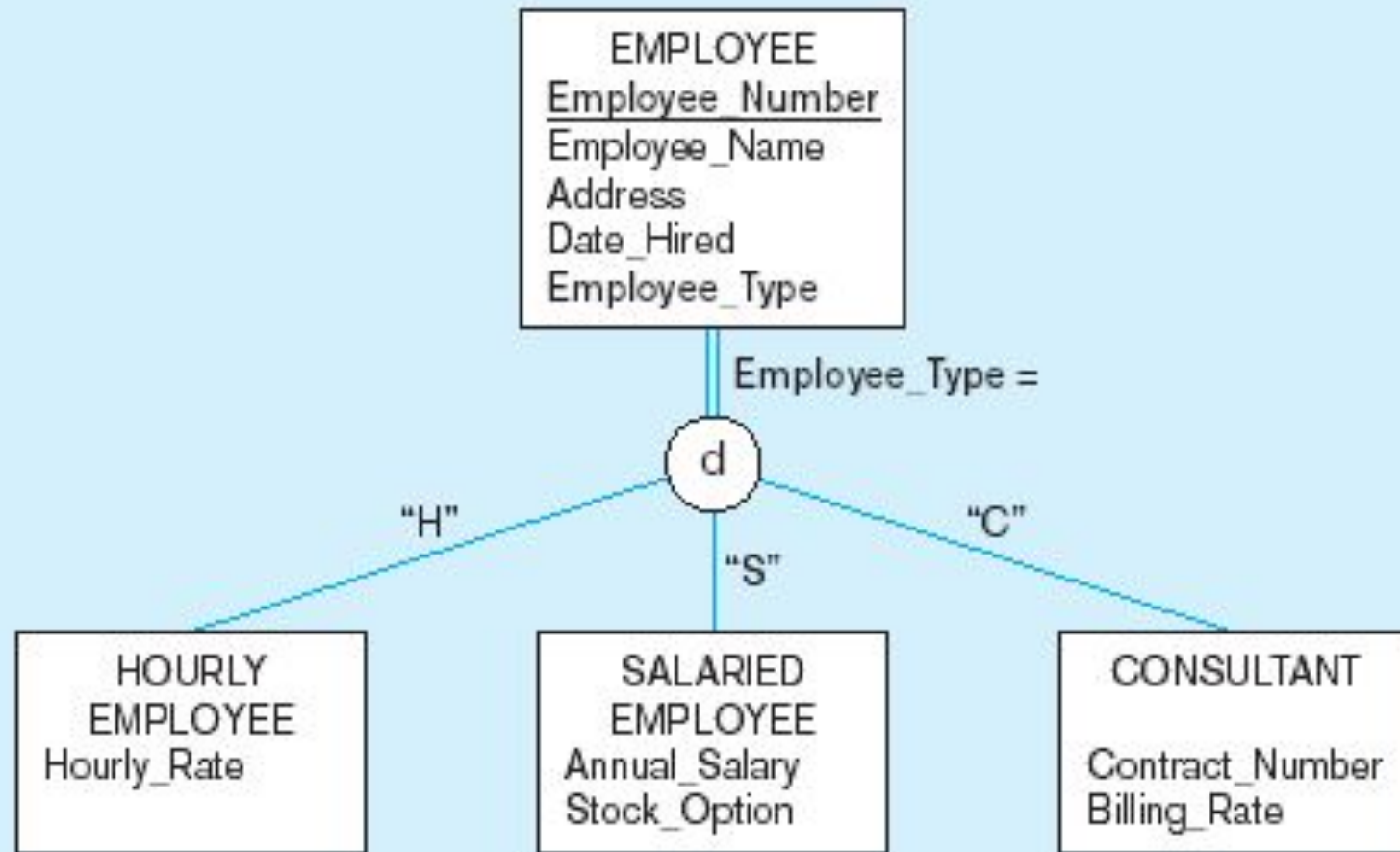
Transforming EER Diagrams into Relations (cont.)

Mapping Supertype/Subtype Relationships

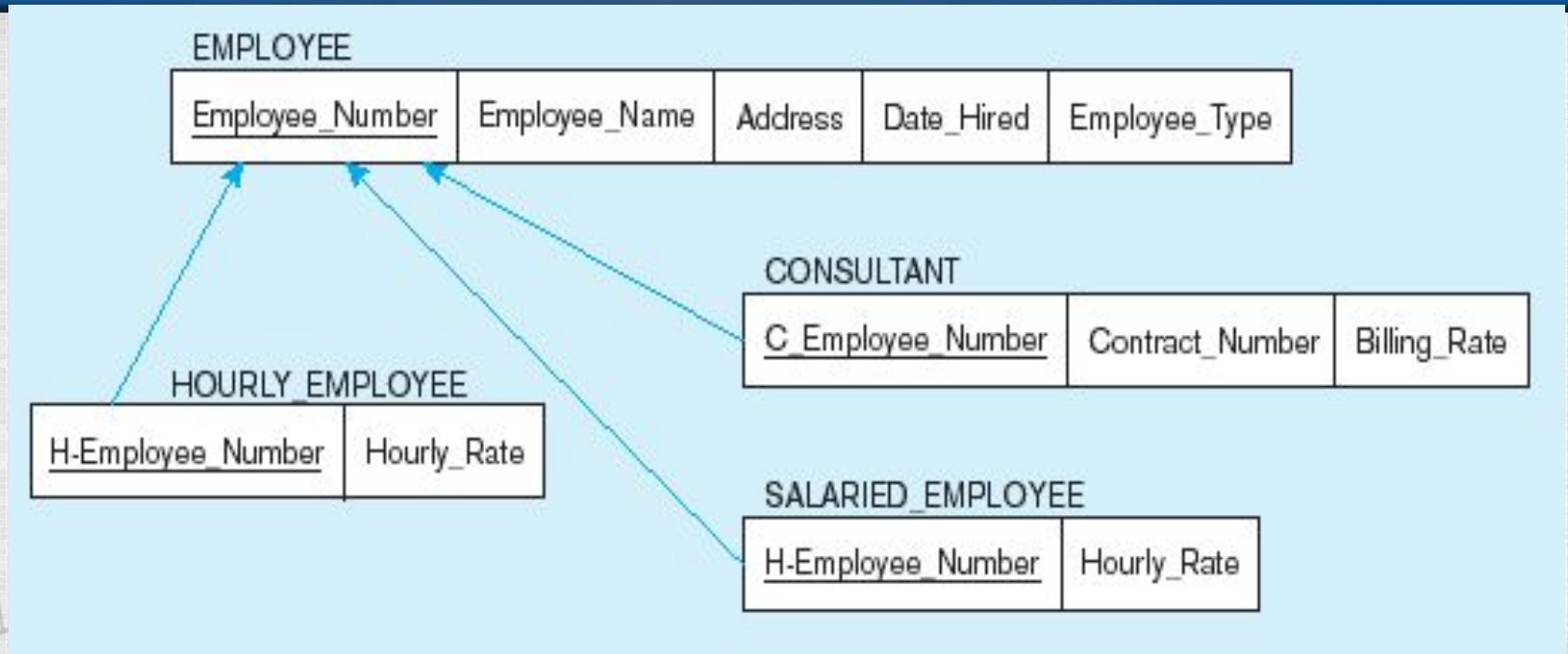
- One relation for supertype and for each subtype
- Supertype attributes (including identifier and subtype discriminator) go into supertype relation
- Subtype attributes go into each subtype; primary key of supertype relation also becomes primary key of subtype relation
- 1:1 relationship established between supertype and each subtype, with supertype as primary table



Supertype/subtype relationships



Mapping Supertype/subtype relationships to relations



These are implemented as one-to-one relationships

